

L14: Python MySQL

RECAP from Lab #13

- Transaction

```
START TRANSACTION;
```

```
INSERT INTO users (id, username)
VALUES (1, 'john');
```

```
UPDATE users
SET email = 'john.doe@example.com'
WHERE id = 1;
```

- Cursor

```
DELIMITER $$
CREATE PROCEDURE create_email_list (
    INOUT email_list TEXT
)
BEGIN
    DECLARE done BOOL DEFAULT false;
    DECLARE email_address VARCHAR(100) DEFAULT "";
    DECLARE cur CURSOR FOR SELECT email FROM employees;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = true;

    OPEN cur;

    SET email_list = '';
    process_email: LOOP
        FETCH cur INTO email_address;
        IF done = true THEN
            LEAVE process_email;
        END IF;
        SET email_list = CONCAT(email_address, ";", email_list);
    END LOOP;

    CLOSE cur;

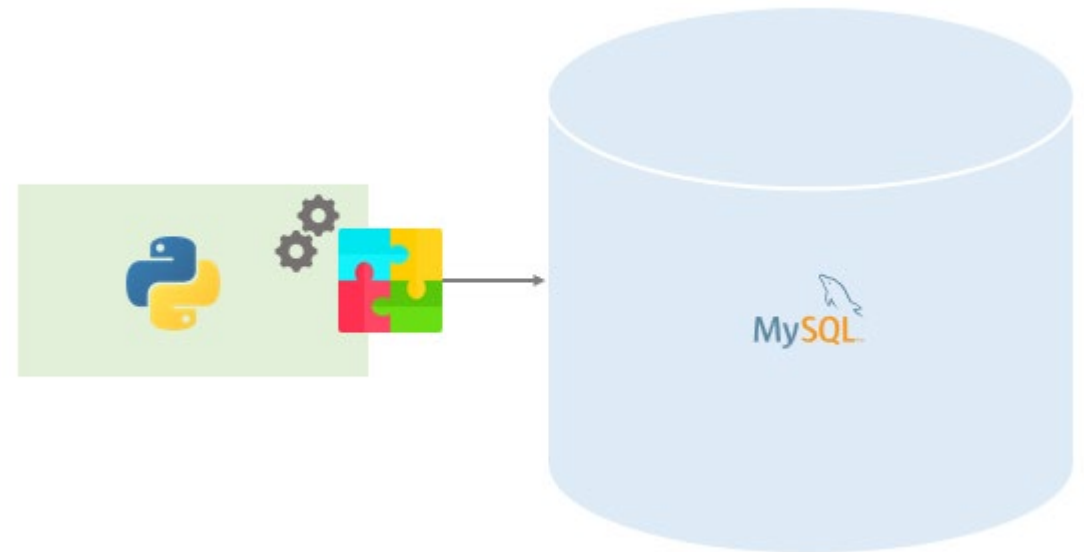
END$$
DELIMITER ;
```

Goal

- Connect to MySQL server from a Python program.
- Insert, Select, Update, and Delete data in the database in Python.
- Call MySQL stored procedures from Python.

Prerequisites

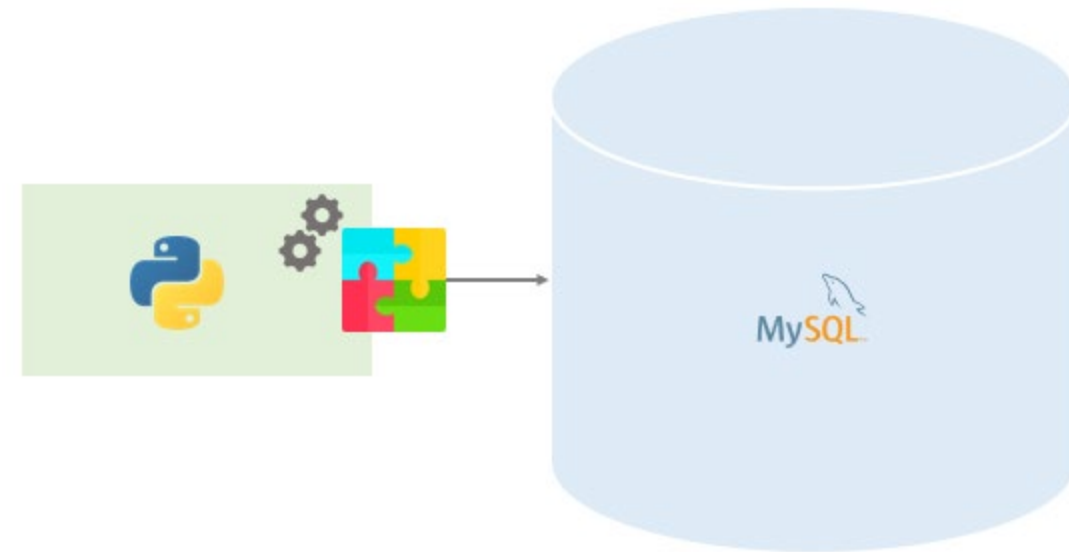
- Python Programming
- Basic MySQL
- MySQL stored procedures



14.1 MySQL Connector/Python

To interact with the MySQL database from Python, you need a database driver. MySQL Connector/Python is a standardized database driver provided by MySQL.

Connector	MySQL Connector/Python
Best Connector Version	9.6.0+
Supported Python	3.10, 3.11, 3.12, 3.13, 3.14
Supported MySQL Server	8.0, 8.4 (LTS), 9.x



Installing MySQL Connector/Python

- Python 3.10+
 - Python `mysql-connector-python` package
- ```
>> pip install mysql-connector-python
```

# 14.2 Check Connection



connect.py

```
import mysql.connector

Initialize a variable to hold the database connection
conn = None

try:
 # Attempt to establish a connection to the MySQL database
 conn = mysql.connector.connect(host='localhost',
 port=3306,
 database='database_name',
 user='<username>',
 password='<password>')

 # Check if the connection is successfully established
 if conn.is_connected():
 print('Connected to MySQL database')

except mysql.connector.Error as e:
 # Print an error message if a connection error occurs
 print(e)

finally:
 # Close the database connection in the 'finally' block to ensure it happens
 if conn is not None and conn.is_connected():
 conn.close()
```



Connected to MySQL  
database

# 14.3 Connect via a config file

 app.ini

```
[mysql]
host = localhost
port = 3306
database = lab14
user = <username>
password = <password>
```

 config.py

```
from configparser import ConfigParser

def read_config(filename='app.ini', section='mysql'):
 config = ConfigParser()
 config.read(filename)

 data = {}
 if config.has_section(section):
 items = config.items(section)
 for item in items:
 data[item[0]] = item[1]
 else:
 raise Exception(f'{section} section not found in the
{filename} file')
 return data

if __name__ == '__main__':
 config = read_config(filename='connection_test/app.ini')
 print(config)
```

 connect.py

```
from mysql.connector import MySQLConnection, Error
from config import read_config

def connect(config):
 """ Connect to MySQL database """
 conn = None
 try:
 print('Connecting to MySQL database...')
 conn = MySQLConnection(**config)

 if conn.is_connected():
 print('Connection is established.')
 else:
 print('Connection is failed.')
 except Error as error:
 print(error)
 finally:
 if conn is not None and conn.is_connected():
 conn.close()
 print('Connection is closed.')

if __name__ == '__main__':
 config = read_config(filename='connection_test/app.ini')
 connect(config)
```

## 14.4 Process Table

1. Connect to the MySQL Database, you get a MySQLConnection object.
2. Create a MySQLCursor object from the MySQLConnection object.
3. Use the cursor to execute a query by calling its execute() method.
  - Use fetchone() , fetchmany() or fetchall() method to fetch data from the result set.
    - Use fetchone() method to retrieve a single row from a result set.
    - Use fetchmany() method to retrieve a specified number of rows from a result set.
    - Use fetchall() method to retrieve all rows from a result set.
4. Close the cursor as well as the database connection by calling the close() method of the corresponding objects.



# Query Data from a Table

```
def query_with_fetchall(config):
 try:
 conn = MySQLConnection(**config)
 cursor = conn.cursor()

 # Querying
 cursor.execute("SELECT * FROM books")

 # Fetch all rows from the result set
 rows = cursor.fetchall()

 print('Total Row(s):', cursor.rowcount)
 for row in rows:
 print(row)

 except Error as e:
 print(e)
 finally:
 cursor.close()
 conn.close()
```



4 • SELECT \* FROM books;

| Result Grid |    |                       |               | Filter Rows: | Edit: | Export/Import: |
|-------------|----|-----------------------|---------------|--------------|-------|----------------|
|             | id | title                 | isbn          |              |       |                |
|             | 1  | Bel and the Dragon    | 123828863494  |              |       |                |
|             | 2  | Daughters of Men      | 1234404543724 |              |       |                |
|             | 3  | The Giant on the Hill | 1236400967773 |              |       |                |
|             | 4  | Marsh Lights          | 1233673027750 |              |       |                |

## Output

```
Total Row(s): 83
(1, 'Bel and the Dragon ', '123828863494')
(2, 'Daughters of Men ', '1234404543724')
(3, 'The Giant on the Hill ', '1236400967773')
(4, 'Marsh Lights ', '1233673027750')
(5, 'Mr. Wodehouse and the Wild Girl ', '1232423190947')
...
```

# SHOW TABLES

```
def show_tables(config):
 try:
 conn = MySQLConnection(**config)
 cursor = conn.cursor()

 # List Tables' name
 cursor.execute("SHOW TABLES")

 # Fetch and display the tables
 print("Tables in the database:")
 for table in cursor:
 print(table[0])

 except Error as e:
 print(e)
 finally:
 cursor.close()
 conn.close()
```



6 • SHOW TABLES;

Result Grid | Filter Rows:

Tables\_in\_lab14

authors

book\_author

books

Output

Tables in the database:  
authors  
book\_author  
books

# Insert / Update / Delete data



## Syntax

```
query = """statement...
 statement...
 statement... """

with MySQLConnection(**config) as conn:
 with conn.cursor() as cursor:

 cursor.execute(query, args)

 conn.commit()
```

Note. args must be in tuple format.

# Insert a row into a table



```
def insert_books(config, title, isbn):
 query = """INSERT INTO books(title,isbn)
 VALUES(%s,%s)"""

 args = (title, isbn)
 try:
 with MySQLConnection(**config) as conn:
 with conn.cursor() as cursor:
 cursor.execute(query, args)
 conn.commit()
 except Error as e:
 print(e)

insert_books(config, 'ABC', '123')
```

| id | title              | isbn          |
|----|--------------------|---------------|
| 81 | The Stereo and God | 1231316672178 |
| 82 | UFO vs. CBS        | 1239778693754 |
| 83 | Vietnam Victory    | 1237098200581 |
| 84 | ABC                | 123           |

# Update Data in a Table



```
def update_book(config, book_id, title):
 query = """ UPDATE books
 SET title = %s
 WHERE id = %s """

 data = (title, book_id)

 try:
 with MySQLConnection(**config) as conn:
 with conn.cursor() as cursor:
 cursor.execute(query, data)
 conn.commit()

 except Error as e:
 print(e)

update_book(config, 37, 'The Giant on the Hill *** TEST ***')
```

| id | title                              | isbn          |
|----|------------------------------------|---------------|
| 36 | Daughters of Men                   | 1235349316660 |
| 37 | The Giant on the Hill *** TEST *** | 1235644620578 |
| 38 | Marsh Lights                       | 1235736344898 |
| 39 | Mr. Wodehouse and the Wild Girl    | 1232744187226 |

# Delete Data from a Table



```
def delete_book(config, book_id):
 query = """ DELETE FROM books
 WHERE id = %s """

 data = (book_id,)

 try:
 with MySQLConnection(**config) as conn:
 with conn.cursor() as cursor:
 cursor.execute(query, data)
 conn.commit()

 except Error as e:
 print(e)

delete_book(config, 38)
```

| id | title                              | isbn          |
|----|------------------------------------|---------------|
| 36 | Daughters of Men                   | 1235349316660 |
| 37 | The Giant on the Hill *** TEST *** | 1235644620578 |
| 39 | Mr. Wodehouse and the Wild Girl    | 1232744187226 |
| 40 | The Fairy Castle                   | 1233729213076 |

# Call Stored Procedures in Python

## Stored Procedures

```
DELIMITER $$

CREATE PROCEDURE find_by_isbn(
 IN p_isbn VARCHAR(13),
 OUT p_title VARCHAR(255)
)
BEGIN
 SELECT title
 INTO p_title
 FROM books
 WHERE isbn = p_isbn;
END$$

DELIMITER ;
```



## Stored Procedures

```
CALL find_by_isbn('1236400967773',@title);
SELECT @title;
```

@title

The Giant on the Hill

# Call Stored Procedures in Python



```
def find_book_by_isbn(config, isbn):
 args = (isbn, 0)

 try:
 with MySQLConnection(**config) as conn:
 with conn.cursor() as cursor:
 result_args = cursor.callproc('find_by_isbn', args)

 book_title = result_args[1]
 return book_title
 except Error as e:
 print(e)

result = find_book_by_isbn(config, '1236400967773')
print(result)
```

Output

The Giant on the Hill